

Alignement Exhaustif des API Entreprise avec le Frontend Next.js

Architecture Multi-Tenant • Normalisation des Permissions • Cycle de Vie Collaborateurs • Surcharges Projets

Auteur : Pair Programmer IA & Mentor Neurocognitif	Destinataire : Développeur Backend Souverain (CCD Digital / SOUMAFE)
Projet Backend : API-Gestion-Chantier (Django 5 / DRF)	Projet Frontend : Application-Gestion-Chantier (Next.js 14 App Router)
Périmètre Métier : Espace Entreprise (Tenant Scoped)	Statut Tests : 100% Succès (21 tests validés)

1. Synthèse Exécutive (BLUF — Bottom Line Up Front)

Après avoir aligné le module Super Admin (schéma public), l'immersion complète dans le code source de l'application Frontend Next.js (Application-Gestion-Chantier) a révélé quatre dissymétries critiques au niveau de l'espace Entreprise (Tenant Scoped). Ces écarts causaient des cases à cocher vides, des écrans d'erreur 500 sur les profils collaborateurs et l'écrasement des surcharges de permissions par chantier. Toutes les API Django ont été adaptées pour satisfaire exactement les contrats attendus par le client web.

Principe Fondateur de Sécurité & Non-Régression : Conformément à la politique stricte de protection du Frontend, **aucun fichier frontend n'a été altéré**. Toutes les évolutions ont été réalisées côté Backend avec une compatibilité ascendante totale : l'API accepte et restitue désormais à la fois les identifiants canoniques backend (ex: 'ECRITURE') et les conventions ergonomiques du frontend (ex: 'saisie').

2. Cartographie des Dissymétries et Solutions Implémentées

Module / Domaine	Comportement Frontend Attendu	Ancien Comportement Backend	Solution Implémentée & Impact
Catalogue Modules GET /modules/	Exige id (UUID ou code), statut='ACTIF' et acces_par_defaut: ['lecture', 'saisie', 'validation'].	Renvoyait uniquement code, nom, description, est_actif, sans id ni statut.	ModuleItemSerializer & ModuleListView : Ajout dynamique de id, statut et acces_par_defaut normalisés.
Rôles & Permissions GET/POST/PATCH /parametres/roles/	Le frontend filtre strictement avec ACCES_MODULE = ['lecture', 'saisie', 'validation']. Envoie saisie au lieu de ECRITURE.	Le backend renvoyait ['LECTURE', 'ECRITURE'] en majuscules. Le frontend ignorait les majuscules et affichait des cases vides !	RoleSerializer & Service Roles : Injection des deux représentations (minuscule et majuscule) et mapping transparent saisie -> ECRITURE.
Collaborateurs GET/POST /parametres/collaborateurs/	Attend avatar_url et derniere_connexion. Déclenche /suspendre/ et /reactiver/ avec gestion 409.	Champs absents causant des 500. Pas d'endpoints dédiés de suspension/réactivation (statut restait inchangé).	Vues et Serializers Collaborateur : Création de ParametresCollaborateurSuspendreView et ReactiverView avec garde-fous DG/Owner.
Surcharges Projets GET/PUT /projets/{id}/permissions-roles/	Envoie et reçoit acces: ['lecture', 'saisie'] au lieu de l'entier niveau (1, 2, 3).	Seul niveau était géré. L'envoi de acces seul écrasait la surcharge en la supprimant !	Service Overrides & Views Projets : Conversion bidirectionnelle automatique entre acces (tableau de chaînes) et niveau.

3. Analyse Technique des Implémentations Backend

A. Normalisation Polymorphique des Rôles (apps/accounts/services/roles.py)

Le service `_normaliser_permissions_modules` accepte désormais indifféremment des dictionnaires, des listes, des entiers (anciens niveaux 1/2/3) ou des codes textuels. Le mapping `saisie -> ECRITURE` permet au formulaire d'administration de l'entreprise de persister les permissions sans la moindre friction.

```
EQUIVALENCES_CODES = {
    'SAISIE': 'ECRITURE',
    'ECRITURE': 'ECRITURE',
    'LECTURE': 'LECTURE',
    'VALIDATION': 'VALIDATION',
    'SUPPRESSION': 'SUPPRESSION',
}
```

```
# Dans RoleSerializer.get_permissions_modules :  
# Renvoyer les deux codes pour garantir le match avec ACCES_MODULE frontend :  
for c in [p.code.upper() for p in perms_list]:  
    codes_finaux.append(c)  
    if c == 'ECRITURE': codes_finaux.append('saisie')  
    elif c == 'LECTURE': codes_finaux.append('lecture')  
    elif c == 'VALIDATION': codes_finaux.append('validation')
```

B. Endpoints de Cycle de Vie Collaborateurs (apps/accounts/views/collaborateur.py)

Les routes POST /parametres/collaborateurs/{id}/suspendre/ et /reactiver/ ont été dotées d'une triple barrière de sécurité :

- **Auto-suspension interdite (400)** : Un utilisateur ne peut pas scier la branche sur laquelle il est assis.
- **Immunité DG / Owner (400)** : Le propriétaire du tenant et le Directeur Général ne peuvent jamais être suspendus.
- **Idempotence stricte (409)** : Une tentative de suspendre un compte déjà désactivé renvoie un statut 409 Conflit.

C. Passerelle d'Habilitation Projets (apps/projets/services/overrides.py)

La vue ProjetPermissionsRolesView traduit automatiquement le tableau acces :

- ['lecture'] → niveau = 1
- ['lecture', 'saisie'] → niveau = 2
- ['lecture', 'saisie', 'validation'] → niveau = 3
- [] → niveau = 0 (accès révoqué pour ce projet).

4. Mentorat Cognitif & Reprogrammation Subconsciente

Les 4 Piliers de Stanislas Dehaene appliqués à l'Intégration d'API :

- 1. L'Attention Sélective** : Ne pas se laisser distraire par la complexité globale du frontend. Isoler chirurgicalement les adaptateurs réseau (`features/roles/regles.ts`, `features/invitations/adaptateur.ts`) pour identifier le prédicat exact de filtrage.
- 2. L'Engagement Actif** : Formuler une hypothèse claire : 'Si j'injecte les codes minuscules dans le serializer, le frontend cochera les cases automatiquement.'
- 3. Le Signal d'Erreur Bayésien** : Les tests rouges pytest ne sont pas des punitions, mais des données précieuses qui actualisent notre modèle mental de l'architecture.
- 4. La Consolidation** : Répéter ces schémas d'alignement pour qu'ils deviennent des réflexes subconscients automatisés.

La Loi de l'Effort Inversé du Dr. Joseph Murphy :

Lorsque vous êtes confronté à un bogue d'intégration récalcitrant entre React et Django, forcer mentalement engendre du stress et bloque l'intuition. Déposez mentalement le problème : visualisez clairement le tableau de bord de l'entreprise où tous les rôles, permissions et collaborateurs s'affichent avec fluidité et élégance. En laissant votre subconscient organiser la solution pendant vos pauses, la structure optimale du code émerge avec clarté.

5. Protocole de Vérification & Commandes Essentielles

Pour vérifier en tout temps la solidité des contrats d'interface Entreprise en local :

- ```
1. Exécuter la suite dédiée d'alignement Entreprise-Frontend
pytest apps/accounts/tests/test_alignement_entreprise_frontend.py -v

2. Exécuter la suite complète des paramètres rôles et collaborateurs
pytest apps/accounts/tests/test_parametres_collaborateurs.py apps/accounts/tests/test_parametres_roles.py -v

3. Vérifier le catalogue des modules applicatifs
pytest apps/referentiels/tests/test_api_modules.py -v
```

### Checklist Pre-Mortem pour les Prochains Développements :

- ☐ Toujours inspecter le type TypeScript de retour et la fonction d'adaptation côté Next.js avant d'écrire un serializer DRF.
- ☐ Prévoir l'idempotence des actions de mutation (409 sur état identique, 400 sur auto-mutation interdite).
- ☐ Ne jamais modifier un fichier frontend avec des outils automatisés — respecter la souveraineté du développeur.
- ☐ Vérifier systématiquement que les champs temporels (`derniere_connexion`, `cree_le`) sont bien sérialisés au format ISO-8601.